

React useRef Hook

Eine vollständige Erklärung mit Beispiel

1. Was ist useRef?

useRef ist ein React Hook, der einen **veränderbaren Referenzwert** zurückgibt, der über Re-Renders hinweg bestehen bleibt – ohne dabei einen neuen Render auszulösen.

Das zurückgegebene Objekt hat immer diese Struktur:

```
{ current: initialValue }
```

Der Wert in **current** kann jederzeit gelesen und geschrieben werden. Anders als bei `useState` löst eine Änderung von **ref.current** keinen Re-Render aus.

2. Wann verwendet man useRef?

Anwendungsfall	Beschreibung
DOM-Zugriff	Direkter Zugriff auf ein HTML-Element (z. B. fokussieren, scrollen)
Werte speichern	Werte über Renders hinweg merken, ohne Re-Render auszulösen
Vorherigen State merken	Den letzten Wert einer Variable für Vergleiche speichern
Timer-IDs speichern	setInterval / setTimeout IDs sicher aufbewahren

3. Syntax

```
import { useRef } from 'react';

const myRef = useRef(initialValue);
// Zugriff auf den Wert:
console.log(myRef.current);
// Wert ändern:
myRef.current = neuerWert;
```

4. Durchgehendes Beispiel – Stoppuhr

Wir bauen eine einfache Stoppuhr. Sie zeigt, wie **useRef** sowohl für **DOM-Zugriff** (das Display-Element) als auch zum **Speichern der Timer-ID** genutzt wird.

Schritt 1 – Imports und State einrichten

```
import { useRef, useState } from 'react';

function Stoppuhr() {
  const [zeit, setZeit] = useState(0); // Sekunden anzeigen
  const [laeuft, setLaeuft] = useState(false);
```

useState steuert die angezeigte Zeit und ob die Uhr läuft. Änderungen an diesen Werten lösen ein Re-Render aus – das ist hier gewollt.

Schritt 2 – useRef für die Timer-ID

```
// Speichert die ID des Intervalls – kein Re-Render nötig
const intervallRef = useRef(null);
```

Die Timer-ID braucht man nur intern, um das Intervall später zu stoppen. Ein Re-Render wäre hier unnötig – deshalb useRef statt useState.

Schritt 3 – Start- und Stopp-Logik

```
const starten = () => {
  if (!laeuft) {
    setLaeuft(true);
    // Timer starten und ID in ref speichern
    intervallRef.current = setInterval(() => {
      setZeit(prev => prev + 1);
    }, 1000);
  }
};

const stoppen = () => {
  clearInterval(intervallRef.current); // ID aus ref lesen
  setLaeuft(false);
};

const zuruecksetzen = () => {
  clearInterval(intervallRef.current);
  setLaeuft(false);
  setZeit(0);
};
```

Schritt 4 – JSX rendern

```
return (
  <div>
    <h2>{zeit} Sekunden</h2>
    <button onClick={starten}>Start</button>
    <button onClick={stoppen}>Stop</button>
    <button onClick={zuruecksetzen}>Reset</button>
  </div>
);
}

export default Stoppuhr;
```

Warum hier useRef? setInterval gibt eine ID zurück. Diese ID muss beim Stoppen wieder verwendet werden. Würde man sie in useState speichern, würde jede Aktualisierung einen unnötigen Re-Render auslösen. Mit useRef bleibt die ID stabil und sauber.

5. useRef vs. useState – Der Unterschied

Merkmal	useRef	useState
Löst Re-Render aus?	Nein	Ja
Wert bleibt bei Re-Render?	Ja	Ja
Typischer Einsatz	DOM, Timer-IDs, interne Werte	UI-relevante Daten
Zugriff auf Wert	ref.current	direkte Variable

6. Zusammenfassung

useRef ist ideal, wenn man einen Wert über Re-Renders hinweg speichern möchte, aber **kein erneutes Rendern** benötigt. Die häufigsten Anwendungsfälle sind der direkte Zugriff auf DOM-Elemente sowie das Speichern von Timer-IDs oder anderen internen Referenzen. Für alles, was die Benutzeroberfläche aktualisieren soll, bleibt **useState** die richtige Wahl.

Merksatz: useRef = Wert merken ohne Neu-Rendern. useState = Wert merken UND Neu-Rendern.